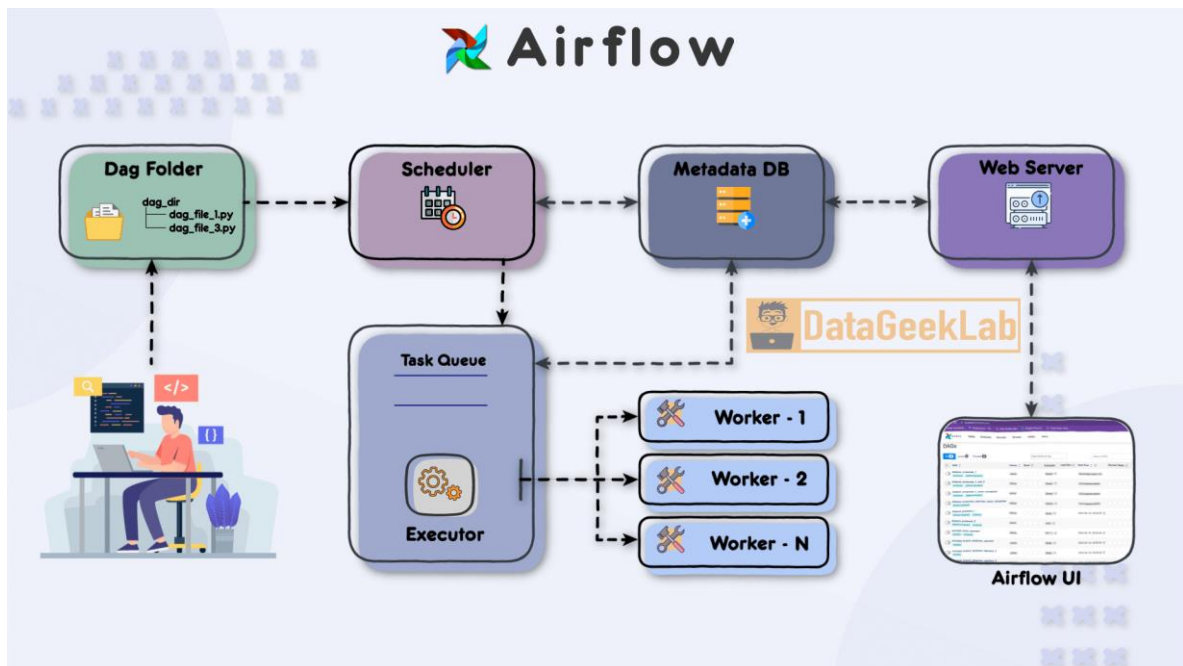


Apache Airflow

1.What is Apache Airflow?

Apache Airflow acts as a centralized "**control tower**" or orchestrator for data engineering workflows. It does not execute data processing or transformations internally. Instead, it schedules and triggers external systems like data warehouses, APIs, and computation engines to run tasks at precise times or under specific conditions.



Key architectural components include:

- **Web Server:** A user interface to trigger, pause, monitor, and debug pipeline executions.
- **Scheduler:** The engine that parses code, manages dependency timing, and hands off ready tasks to workers.
- **Worker:** The actual compute nodes that carry out the assigned tasks.
- **Metadata Database:** A relational store (e.g., PostgreSQL) that tracks task status, user permissions, and job history. [[1](#), [2](#), [3](#), [4](#), [5](#)]

2.Why Choose Airflow Instead of Azure Data Pipelines?

While Azure Data Pipelines are effective for native Microsoft environments, Apache Airflow provides unmatched flexibility for modern data infrastructure.

Use Azure Data Factory (ADF) if:	Use Apache Airflow if:
You want a Visual No-Code GUI	You want Python Code-First
You operate 100% inside Azure	You use Multi-Cloud (AWS/GCP)
You need simple, fast setups	You require custom logic/loops

- **Code-First vs. UI-Driven:** Azure relies on a visual interface. Airflow workflows are written entirely in Python, enabling software practices like Git version control, continuous integration (CI/CD), and automated testing.
 - **Multi-Cloud Agility:** Azure Pipelines lock you into the Microsoft ecosystem. Airflow features a massive open-source ecosystem of plug-and-play operators to seamlessly bridge AWS, Google Cloud, Snowflake, and regional infrastructure together.
 - **Dynamic Pipeline Generation:** Airflow allows for dynamic task creation. For instance, a single Python for loop can automatically generate 50 separate parallel tasks based on a live API response. Azure requires rigid, pre-defined structural boxes that make looping configurations cumbersome and costly.
 - **Complex Error Recovery:** Airflow provides fine-grained control over task failures. You can design data-aware sensors, precise conditional paths, and localized retry structures that standard serverless functions or simple loops cannot natively accommodate.
-

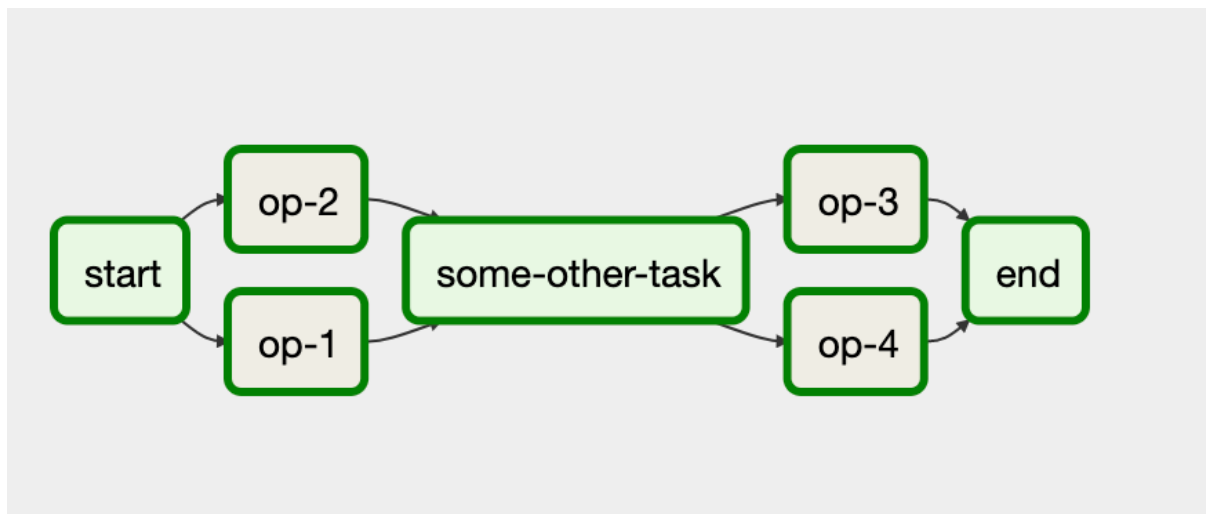
3.What is a DAG?

A **DAG (Directed Acyclic Graph)** is the conceptual map of an Airflow pipeline. In data systems, it defines exactly how tasks are linked together.

The concept breaks down into three distinct mathematical rules:

1. **Directed:** The workflow has a strict, one-way structural direction (Task A must explicitly finish before Task B can start).
2. **Acyclic:** The pipeline cannot contain loops or cycles. Task C cannot point back to Task A, eliminating the danger of infinite running loops.
3. **Graph:** The structure maps individual nodes (individual tasks) against their edges (the dependency relationships connecting them).

Example DAG Visual Flow



In a standard Python DAG script, engineers use operators to declare these tasks—such as an `S3ToSnowflakeOperator` to move data—and explicitly set their order using the bitshift operator (`>>`), like `task_a >> [task_b, task_c] >> task_d`.